FraudShield AI – Explainable credit card fraud detection and risk scoring

The domain of the Project:

Artificial Intelligence & Machine Learning

Team Mentor Designation:

Ms. Akshara A
FSD & AI/ML Trainer & Volunteer
SURE Trust

Team Member:

Surendra Reddy

Period of the project

September 15, 2026 to November 15, 2026

The project titled “FraudShield AI – Explainable credit card fraud detection and risk scoring” has been mentored by Ms. Akshara A, organised by SURE Trust, from May 15, 2026 to November 15, 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Surendra Reddy – _____ Signature

Ms. Akshara A
FSD & AI/ML Trainer & Volunteer
SURE Trust

Prof. Radhakumari
Executive Director & Founder
SURE Trust

Table of contents

1. Executive summary 4
2. Introduction 5
3. Project Objectives 6
4. Methodology & Results 7
5. Social / Industry relevance of the project 13
6. Learning & Reflection 14
7. Future Scope & Conclusion 15

Executive Summary

Credit card fraud causes direct financial loss and weakens trust in digital payments, yet it is very hard to detect: in the dataset used here only 492 of 284,807 transactions (0.17%) are fraudulent, so a model that always answers “legit” is 99.83% accurate while catching no fraud at all. Models in this project are therefore judged by precision, recall and PR-AUC, not accuracy.

FraudShield AI is an explainable fraud detection and risk scoring system. It was trained in Google Colab on the public Kaggle *Credit Card Fraud Detection* dataset using a stratified 80/20 split. Logistic Regression, Random Forest and XGBoost were compared with two ways of handling imbalance, SMOTE and class weights, and a plain XGBoost without any handling was kept as a control. **XGBoost with** `scale_pos_weight` gave the best balance, with a precision of 0.893, recall of 0.847, F1-score of 0.869 and PR-AUC of 0.885 on a test set of 56,962 transactions containing 98 frauds. It missed 15 frauds and raised 10 false alarms.

The model outputs a 0–100 risk score that is mapped to three bands with an action (Low: approve, Medium: review, High: block); the fraud rate rises from 0.02% in the Low band to 89.25% in the High band. SHAP explains every decision by showing the five features that pushed the score. Everything is delivered in a **Streamlit** dashboard where the user picks a real held-out transaction and sees its score, band, action and reasons. Main limitations are the small number of test frauds and 12 frauds that look completely normal and cannot be separated by any model.

Introduction

Background and context of the project

Digital card payments are growing every year and so is card fraud. Banks process far too many transactions for manual checking, so they rely on automated models that score each transaction within moments. A good system must catch fraud without blocking genuine customers, because every false alarm annoys a real person and every missed fraud costs money.

Problem statement or goals of the project

Given the details of a card transaction, estimate how likely it is to be fraud and explain why. The main difficulty is **class imbalance**, since fraud is only 0.17% of the data. The goals are to compare models and imbalance techniques fairly, to choose a final model using suitable metrics, to turn its output into an understandable risk score with a suggested action, and to deliver it in a simple dashboard.

Scope and limitations of the project

Scope: exploratory analysis, stratified train–test split, scaling, comparison of three models with SMOTE and class weights (plus a no-handling control), risk bands, SHAP explanations and a Streamlit app that works on a sample of real held-out transactions.

Limitations:

- The test set holds only 98 frauds, so small differences between models are not reliable.
- The decision cut-off was explored on the test set, so the reported scores are slightly optimistic.
- Twelve test frauds receive a very low score because their features look normal; they are missed.
- V1–V28 are anonymised (PCA), so features can be ranked but not interpreted in business terms.
- The app is a demo on a sample file, not a real-time service.

Innovation component in the project

Instead of assuming that SMOTE helps, it was tested against class weights and against no handling, and class weighting won. The project goes beyond a yes/no prediction: it gives a 0–100 risk score, three action bands validated against real fraud rates, and a per-transaction SHAP explanation shown in plain language inside the dashboard.

Project Objectives

Clearly defined objectives and goals of the project

- Analyse the transaction data and show how fraud differs from legitimate activity.
- Split and scale the data correctly so the test set stays realistic and free of leakage.
- Compare Logistic Regression, Random Forest and XGBoost with SMOTE and class weights, with a plain XGBoost as a control.
- Judge models by precision, recall, F1-score and PR-AUC instead of accuracy.
- Convert predictions into a risk score and risk bands with a suggested action.
- Explain individual decisions using SHAP.
- Deliver the result as a Streamlit dashboard and publish the code on GitHub.

Expected outcomes and deliverables

- A Colab notebook with the full analysis and model comparison.
- Saved model, scaler and feature list (`fraud_model.pkl`, `scaler.pkl`, `feature_names.pkl`).
- A sample of 398 held-out transactions for the demo (`sample_transactions.csv`).
- A working Streamlit app with risk score, band, action and explanation.
- The GitHub repository (https://github.com/SurendraReddy7468/FraudShield-AI), a project review presentation and this report.

Methodology and Results

Methods/Technology used

Pipeline. EDA → stratified split → imbalance handling → model training → explanation with SHAP and risk score → Streamlit app.

1. Exploratory data analysis. The dataset has no missing values. *Amount*: the median fraud is 9.25 against 22.00 for legitimate transactions, so most fraud is small and amount alone cannot detect it. *V-features*: V17, V14, V12 and V10 have the strongest negative correlation with fraud (-0.33, -0.30, -0.26, -0.22) and V11 the strongest positive ($+0.15$); the fraud and legit curves still overlap, so some fraud looks normal. *Time*: legitimate transactions follow a daily rhythm while fraud comes in bursts and continues at night.

2. Split and scaling. A stratified 80/20 split with `random_state=42` gave 227,845 training transactions (394 frauds) and 56,962 test transactions (98 frauds). `RobustScaler` was fitted on the training data only and applied to *Amount* and *Time*.

3. Imbalance handling. SMOTE was applied after the split and on training data only, creating a 50/50 training set (227,451 per class) while the test set stays real. Class weighting used `class_weight='balanced'` for Logistic Regression, `'balanced_subsample'` for Random Forest and `scale_pos_weight` = 577.29 (the ratio of legit to fraud in training) for XGBoost. A plain XGBoost with no handling was kept as a control.

4. Models. Logistic Regression (baseline), Random Forest (100 trees) and XGBoost (300 trees, depth 6, learning rate 0.1). Predictions use a probability cut-off of 0.5.

5. Evaluation metrics.

- *Precision*: of the transactions flagged as fraud, how many really are fraud.
- *Recall*: of all real frauds, how many were caught.
- *F1-score*: the harmonic mean of precision and recall.
- *PR-AUC*: area under the precision–recall curve, the fair comparison when fraud is rare. ROC-AUC was 0.95–0.98 for every model, even Logistic Regression, so it does not separate them.

Tools/Software used

Python, Pandas, NumPy, Scikit-learn 1.6.1, XGBoost 3.4.1, imbalanced-learn, SHAP 0.54.1, Matplotlib, Seaborn and Streamlit. Google Colab was used for training; VS Code and GitHub for the app and version control.

Data collection approach

The data is the public Kaggle dataset *Credit Card Fraud Detection* (mlg-ulb on Kaggle): 284,807 card transactions made over 48 hours, with 31 columns. V1–V28 are PCA-anonymised features, *Time* and *Amount* are the only original columns, and *Class* is the label (0 legitimate, 1 fraud). There are 492 frauds (0.17%) and no missing values; the notebook checks the shape, missing values and fraud count before use.

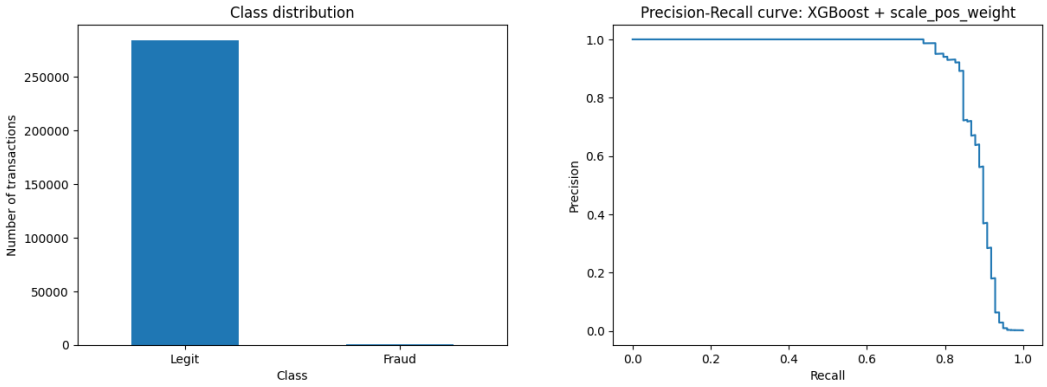


Figure 1: Left: class distribution (fraud is 0.17%). Right: precision–recall curve of the final model.

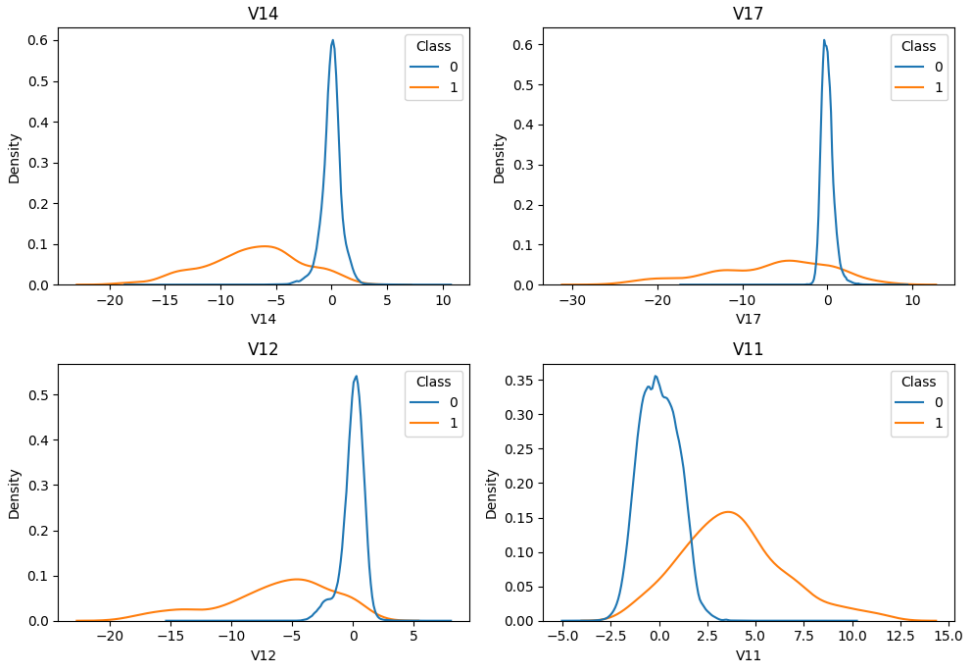


Figure 2: Distribution of V14, V17, V12 and V11 for legitimate and fraud transactions.

Project Architecture

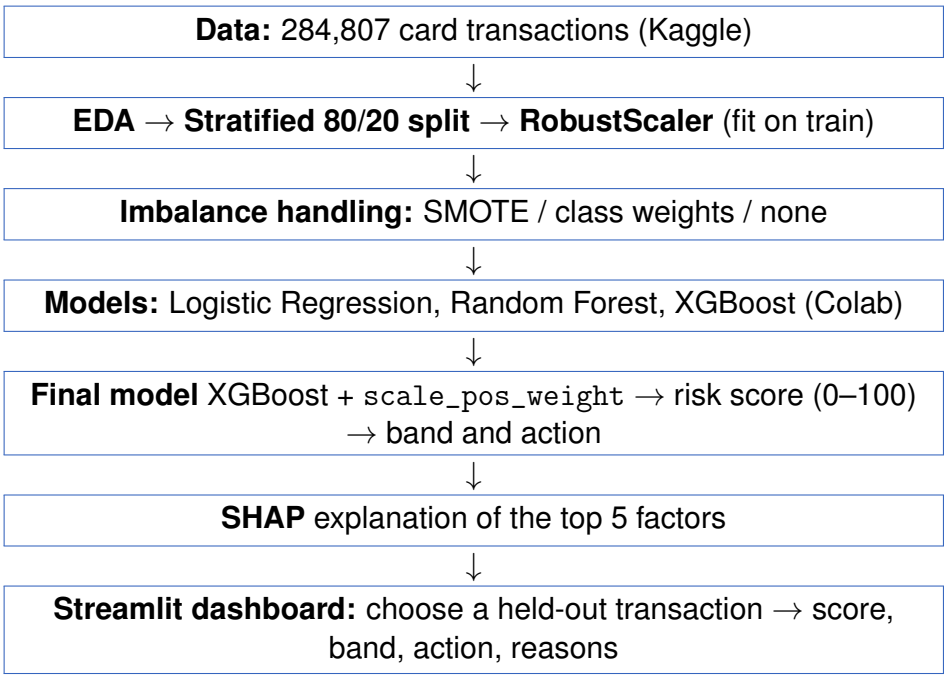


Figure 3: Architecture of FraudShield AI.

Training happens offline in Colab. The model (`fraud_model.pkl`), scaler (`scaler.pkl`) and feature list (`feature_names.pkl`) are saved with joblib. The Streamlit app loads them together with `sample_transactions.csv`, a sample of 398 real held-out transactions (all 98 test frauds and 300 random legitimate ones), so the app only shows data the model never trained on.

Results

Was SMOTE worth it? Three XGBoost variants were compared:

Table 1: XGBoost with different imbalance handling (test set).

Variant	F1	PR-AUC	False alarms
No handling	0.862	0.849	5
SMOTE (50/50)	0.793	0.871	30
scale_pos_weight	**0.869**	**0.885**	10

SMOTE trains on 50/50 data, so the model over-flags real data where fraud is only 0.17%. Class weighting gave the best F1 and PR-AUC and was deployed.

Full model comparison:

Table 2: Performance of all models on the test set (56,962 transactions, 98 frauds).

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
LogReg + SMOTE	0.059	0.918	0.111	0.971	0.725
Random Forest + SMOTE	0.845	0.837	0.841	0.973	0.870
XGBoost + SMOTE	0.737	0.857	0.793	0.980	0.871
LogReg + class_weight	0.061	0.918	0.114	0.972	0.718
Random Forest + class_weight	0.960	0.735	0.832	0.948	0.860
XGBoost + scale_pos_weight	**0.893**	**0.847**	**0.869**	0.967	**0.885**
XGBoost plain	0.940	0.796	0.862	0.963	0.849

Tree models beat Logistic Regression clearly: Logistic Regression catches 91.8% of fraud but its precision is about 0.06, which means roughly 1,400 false alarms. The final model has the highest F1 and PR-AUC.

Table 3: Confusion matrix of the final model (XGBoost + `scale_pos_weight`).

	Predicted legitimate	Predicted fraud
Actual legitimate	56,854	10
Actual fraud	15	83

The final model catches 83 of 98 frauds (recall 0.847) and only 10 of the 93 transactions it flags are false alarms (precision 0.893).

Choosing the cut-off. Moving the cut-off changes the balance between caught fraud and false alarms:

Table 4: Effect of the probability cut-off on the final model.

Cut-off	Precision	Recall	Flagged
0.1	0.822	0.847	101
0.3	0.874	0.847	95
0.5	0.892	0.847	93
0.7	0.891	0.837	92
0.9	0.931	0.827	87

Risk score and bands. The risk score is the predicted probability multiplied by 100. It is a ranking, not a true probability. Splitting it into three bands gives actions that match the real fraud rate:

Table 5: Risk bands on the test set.

Band	Probability	Transactions	Fraud rate	Action
Low	below 0.02	56,834	0.02%	Approve
Medium	0.02 to 0.5	35	8.57%	Review
High	0.5 or above	93	89.25%	Block

The fraud rate rises about 4,500 times from the Low to the High band. In the High band 83 of 93 transactions are real fraud.

SHAP explanation. The strongest drivers of the model are V14, V4, V12, V10 and V11. Low values of V14, V12 and V10 push the score toward fraud. Because these features are anonymised, the app shows which features matter but not what they mean.

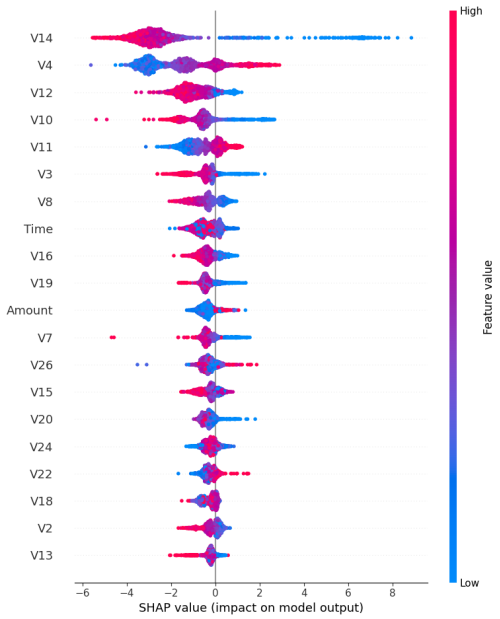


Figure 4: SHAP summary: which features push the score toward fraud or legit.

Final project working screenshots along with supporting explanation

The Streamlit app lets the user pick a held-out transaction (all, fraud only or legit only) and shows the risk score, risk band, suggested action, the true label (for demo only) and the top five reasons with a SHAP chart.

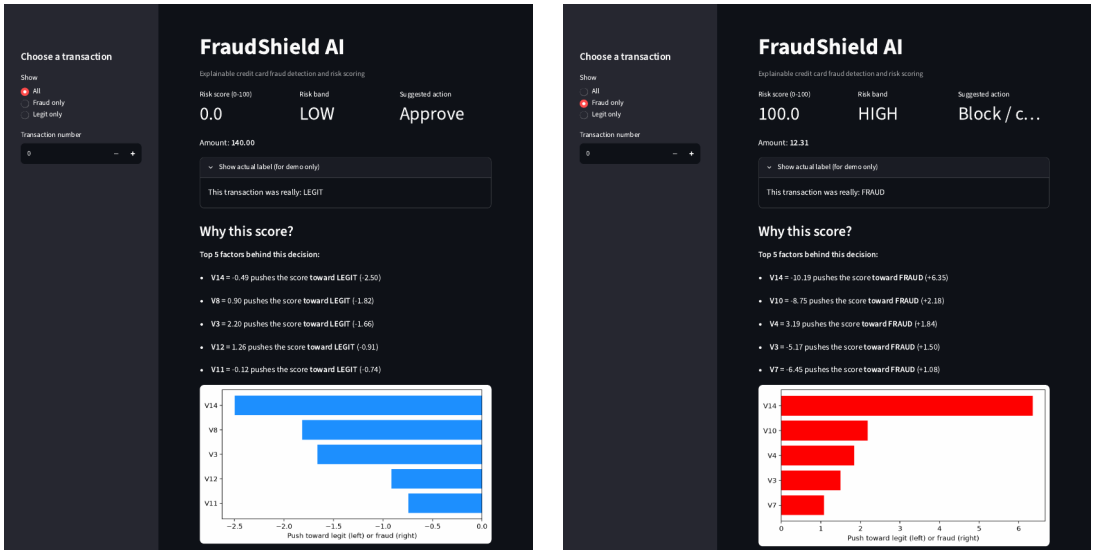


Figure 5: Left: a legitimate transaction. Right: a fraudulent transaction.

Case 1, legitimate transaction. Amount 140.00 receives a risk score of 0.0, band LOW and the action Approve. V14 = -0.49 and the other values look normal, so every factor pushes toward legit.

Case 2, fraud caught. Amount 12.31 receives a risk score of 100.0, band HIGH and the action Block. V14 = -10.19 (+6.35) and V10 = -8.75 (+2.18) lead the explanation, followed by V4, V3 and V7.

Case 3, fraud missed. A real fraud scored 3, band LOW, and was approved by mistake. Its features look normal, so no model could separate it. This is documented as a limitation: 12 of the 98 test frauds fall in the Low band.

How to run the app:

```
pip install -r requirements.txt
streamlit run app.py
```

Project GitHub Link

https://github.com/SurendraReddy7468/FraudShield-AI

Social / Industry Relevance of the Project

Industry relevance. Banks, fintech companies and payment gateways need fast, automated fraud screening that is also explainable, because regulators and customers ask why a transaction was blocked. The three-band approach (approve, review, block) matches how real fraud teams work: only the small Medium band needs a human, which reduces manual workload. The project also shows an important industry lesson, that accuracy is meaningless on rare events and that metrics such as PR-AUC must be used.

Social relevance. Fraud hurts ordinary customers, and new and first-time digital payment users, including many in rural areas, are especially vulnerable. Detecting fraud while rarely blocking genuine payments (only 10 false alarms among 56,864 legitimate test transactions) builds trust in digital payments and supports financial inclusion.

Learning and Reflection

New learnings of the team member

Surendra Reddy

- *Technology:* why accuracy misleads on imbalanced data; stratified splitting; fitting the scaler on training data only to avoid leakage; applying SMOTE after the split and on training data only; class weights and `scale_pos_weight`; reading precision, recall, F1, ROC-AUC and PR-AUC; choosing probability cut-offs; turning probabilities into risk bands; explaining models with SHAP; saving models with joblib; building a Streamlit app.
- *Management and professional skills:* testing ideas instead of assuming them (SMOTE versus class weights versus nothing), documenting limitations honestly, presenting results to my guide, and using Git and GitHub for version control.

Overall experience

Working on FraudShield AI taught me that the choice of metric matters as much as the choice of model. The most surprising result was that every model, even Logistic Regression, had a ROC-AUC between 0.95 and 0.98, while their precision and PR-AUC were very different. I also learned that a popular technique such as SMOTE does not always help: here it made the model over-flag real data. The hardest part was accepting that some frauds look completely normal and can never be caught, and explaining that clearly instead of hiding it. Moving the model from a Colab notebook into a dashboard that explains each decision showed me how a machine learning result becomes something a non-technical person can use.

Conclusion and Future Scope

Recap objectives and achievements

- The data was analysed and split without leakage; fraud was shown to be small in amount and visible mainly in V14, V17, V12 and V10.
- Three models were compared with SMOTE and class weights, plus a no-handling control, and SMOTE was shown not to help here.
- The final XGBoost model with `scale_pos_weight` reached precision 0.893, recall 0.847, F1-score 0.869 and PR-AUC 0.885, catching 83 of 98 test frauds with only 10 false alarms.
- A 0–100 risk score with Low, Medium and High bands turned predictions into actions, with fraud rates of 0.02%, 8.57% and 89.25%.
- SHAP explanations and a Streamlit dashboard make each decision understandable, and the code is published on GitHub.

Future scope of this project

- Use cross-validation and a separate validation set to choose the cut-off, so the test scores are not optimistic.
- Calibrate the probabilities so the risk score becomes a true probability.
- Add feature engineering such as the hour of the day.
- Replace the demo sample with a real-time prediction API.
- Test on newer and larger data to check that the model generalises.

References

[1] Machine Learning Group – ULB, *Credit Card Fraud Detection*, Kaggle dataset. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proc. ACM SIGKDD*, 2016. https://arxiv.org/abs/1603.02754

[3] N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique," *JAIR*, vol. 16, 2002. https://doi.org/10.1613/jair.953

[4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *NeurIPS*, 2017. https://arxiv.org/abs/1705.07874

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[6] Streamlit documentation, https://docs.streamlit.io

[7] FraudShield AI source code and notebook, https://github.com/SurendraReddy7468/FraudShield-AI